

Τελική Εργασία 2026 Σεπτέμβριος
Ενσωματωμένα Συστήματα Πραγματικού Χρόνου
Ξάνθος Παναγιώτου
AEM: 11114

Οι μετρήσεις έγιναν την μέρα Κυριακή 20/09/2026.

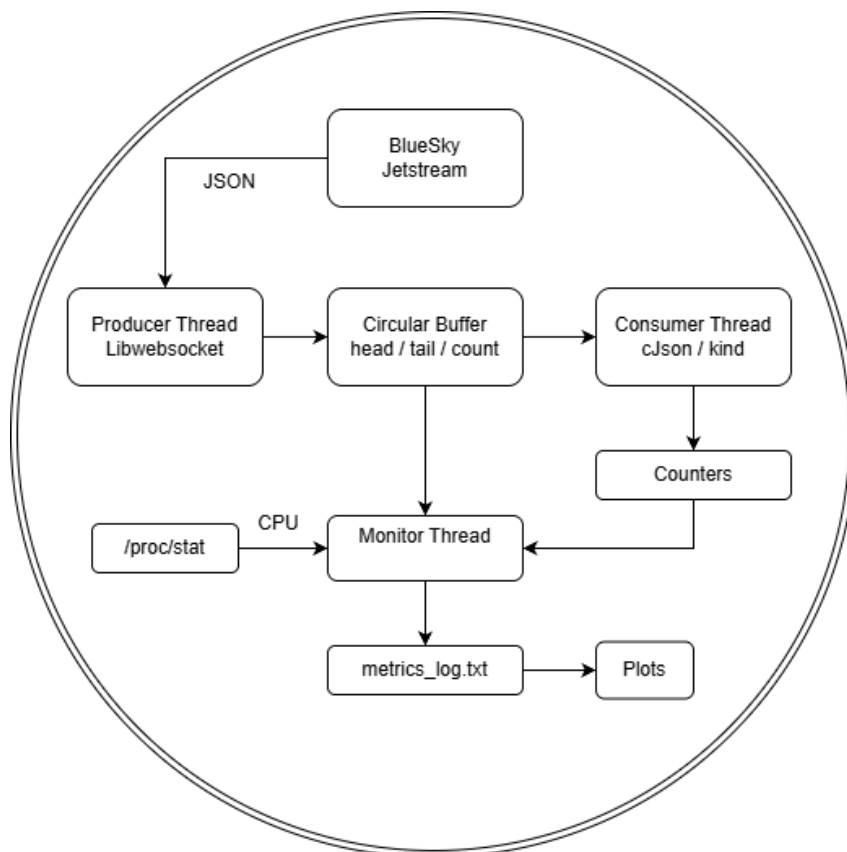
GitHub Link: <https://github.com/xanthosp/Real-Time-Embedded-Systems-September-2026.git>

1. Σκοπός και αρχιτεκτονική συστήματος

Στόχος της εργασίας ήταν η υλοποίηση ενός πολυνηματικού συστήματος σε C πάνω σε φυσικό Raspberry Pi Zero W, το οποίο λαμβάνει σε πραγματικό χρόνο JSON μηνύματα από το Bluesky Jetstream μέσω WebSocket, τα επεξεργάζεται ασύγχρονα και καταγράφει ανά δευτερόλεπτο μετρικές λειτουργίας. Η σχεδίαση ακολουθεί το μοντέλο Producer-Consumer με τρία POSIX threads: Producer, Consumer και Monitor.

Ο Producer είναι το δικτυακό, event-driven νήμα. Η libwebsockets καλεί το WebSocket_callback() όταν δημιουργείται σύνδεση, όταν φτάνει νέο frame ή όταν χαθεί η σύνδεση. Επειδή ένα WebSocket μήνυμα μπορεί να φτάσει σε περισσότερα από ένα fragments, τα τμήματα ενώνονται πρώτα σε στατικό buffer μεγέθους Message Size=50000 bytes και μόνο όταν ληφθεί το τελικό fragment το πλήρες JSON περνά στον κυκλικό buffer μέσω της συνάρτησης queueAdd().

Ο κυκλικός buffer έχει Queue Size=100 θέσεις. Οι μεταβλητές head, tail και count κρατούν αντίστοιχα την επόμενη θέση εγγραφής, την επόμενη θέση ανάγνωσης και το πλήθος των αποθηκευμένων μηνυμάτων. Η μνήμη του buffer είναι στατικά δεσμευμένη, οπότε δεν γίνεται malloc/free για κάθε εισερχόμενο μήνυμα. Αν ο buffer είναι πλήρης δεν γίνεται overwrite δεδομένων. Στην τελική 24ωρη μέτρηση η μέγιστη παρατηρούμενη πληρότητα ήταν 7%, επομένως το όριο των 100 θέσεων ήταν ικανοποιητικό για μέγεθος του Buffer.



Σχήμα 1. Αρχιτεκτονική της εφαρμογής και ροή δεδομένων μεταξύ των τριών νημάτων.

2. Υλοποίηση, συγχρονισμός και περιοδική καταγραφή

Ο Consumer καλεί την `queueDel()` και, όταν ο buffer είναι άδειος, περιμένει στην condition variable `notEmpty`. Έτσι αποφεύγεται το busy waiting και το νήμα ξυπνά μόνο όταν ο Producer καλέσει το `signal()`. Το JSON γίνεται parse με `cJSON` και ελέγχεται το πεδίο `kind`. Ανάλογα με την τιμή του αυξάνεται ένας από τους μετρητές `commit`, `identity`, `account` ή `info`. Το parse γίνεται εκτός του critical section και το mutex κρατιέται μόνο για την ενημέρωση των κοινών counters, ώστε ο χρόνος αποκλεισμού να παραμένει μικρός.

Η πρόσβαση τόσο στον circular buffer όσο και στους global counters προστατεύεται από κοινό `pthread_mutex`. Με αυτόν τον τρόπο Producer, Consumer και Monitor δεν μπορούν να μεταβάλουν τα ίδια κοινά δεδομένα ταυτόχρονα, οπότε αποφεύγονται και τυχόν race conditions. Στο Monitor, οι counters αντιγράφονται σε τοπικές μεταβλητές και μηδενίζονται μέσα στο ίδιο κλείδωμα. Η εγγραφή στο αρχείο και ο υπολογισμός CPU γίνονται αφού απελευθερωθεί το mutex.

Το Monitor εκτελείται μία φορά ανά δευτερόλεπτο με `clock_nanosleep(Clock real time, Timer Abs time)`. Η αρχική προθεσμία τοποθετείται στο επόμενο ακέραιο δευτερόλεπτο και σε κάθε επανάληψη αυξάνεται ακριβώς κατά 1 s. Έτσι μια μεμονωμένη καθυστέρηση δεν μετατοπίζει όλες τις επόμενες εκτελέσεις και περιορίζεται το clock drift. Η πραγματική χρονική στιγμή αφύπνισης λαμβάνεται με `clock_gettime()` και μαζί με τους counters, την πληρότητα του buffer και τη χρήση CPU γράφεται με `append` στο `metrics_log.txt`. Ενώ η CPU υπολογίζεται από τις διαφορές `total` και `idle jiffies` της πρώτης γραμμής του `/proc/stat` μεταξύ δύο διαδοχικών δειγμάτων.

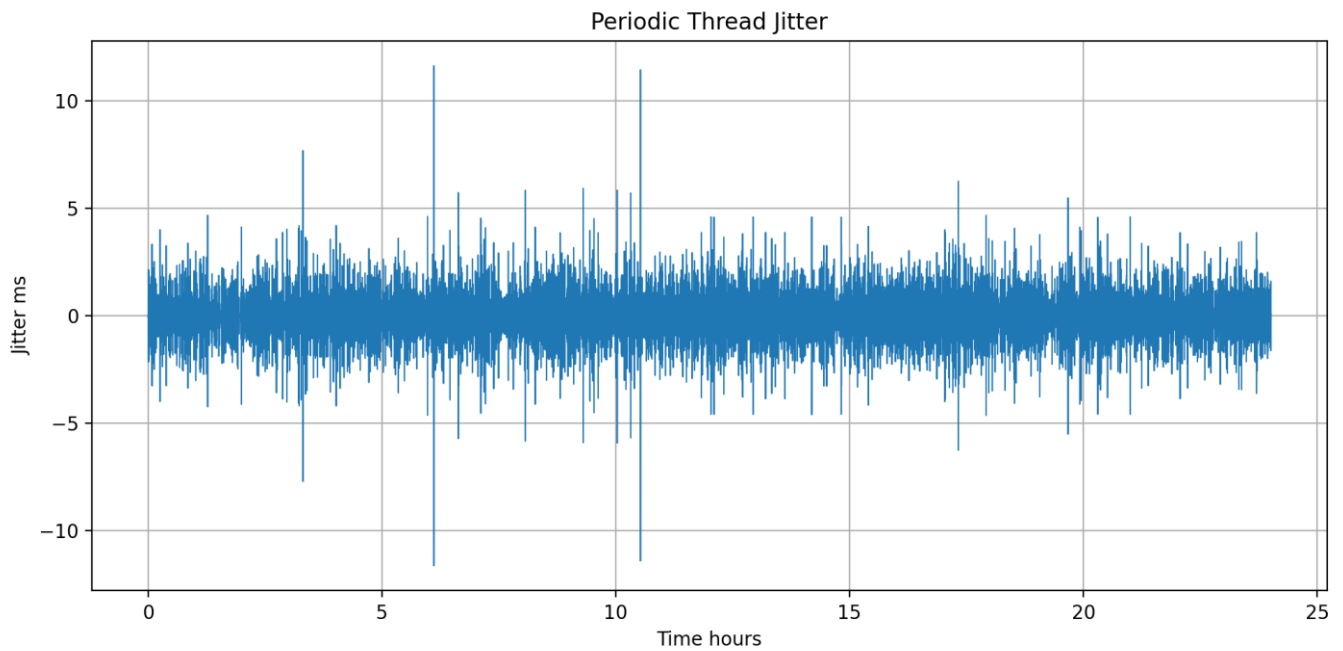
3. Αποτελέσματα 24ωρης εκτέλεσης

Το `metrics_log.txt` περιέχει ακριβώς 86.400 εγγραφές, δηλαδή μία μέτρηση για κάθε δευτερόλεπτο των 24 ωρών. Συνολικά επεξεργάστηκαν 3.394.408 μηνύματα: 3.339.531 `commit`, 24.577 `identity`, 30.300 `account` και 0 `info`. Η επεξεργασία έγινε σε Python με `pandas/NumPy/Matplotlib` για τα σχήματα και τις μετρήσεις των αποτελεσμάτων. Ο συνολικός ρυθμός ανά δευτερόλεπτο υπολογίστηκε ως το άθροισμα των τεσσάρων counters και το signed jitter.

Για το jitter χρησιμοποιήθηκε η απόκλιση κάθε διαδοχικού διαστήματος από το ιδανικό 1 s. Η μέση τιμή είναι σχεδόν μηδενική ($-3,8 \cdot 10^{-7}$ ms), με τυπική απόκλιση 0,470 ms. Παρατηρούνται λίγες μεμονωμένες αιχμές έως περίπου $\pm 11,63$ ms, χωρίς όμως συσσώρευση σφάλματος. Δεν υπάρχει κανένα κενό δευτερολέπτου στα timestamps και η συνολική απόκλιση από το ιδανικό χρονοδιάγραμμα στο τέλος των 24 ωρών είναι περίπου $-0,033$ ms, κάτι που επιβεβαιώνει την αποτελεσματικότητα του absolute-time scheduling.

Metrics Table

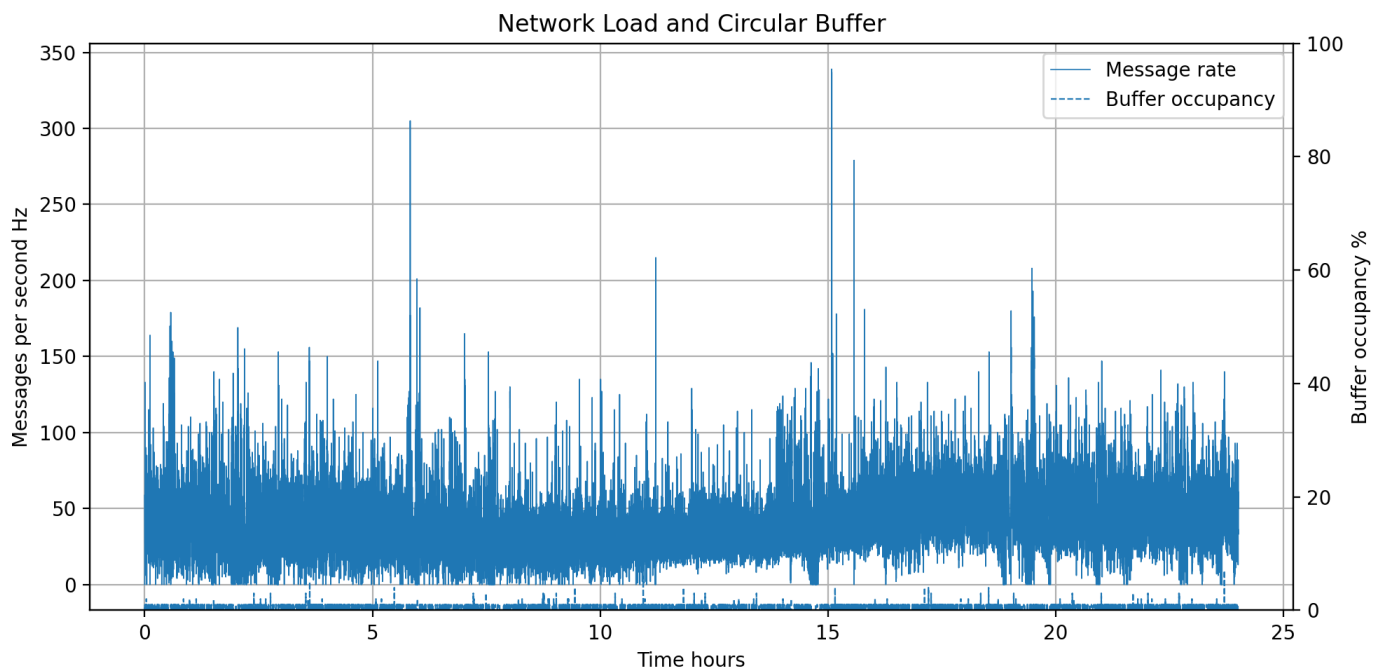
Metrics	Value
Hz (Average, Min, Max)	~39.29, 0, 339/s
Max Buffer	7%
CPU (Average, Max)	~9.1%, 100%
Jitter (σ , Min, Max)	~0,470 ms, -11,628 ms, +11,625 ms
Κενά στα timestamps	0



Σχήμα 2. Jitter του περιοδικού Monitor κατά τη διάρκεια των 24 ωρών.

3.1 Φόρτος δικτύου και circular buffer

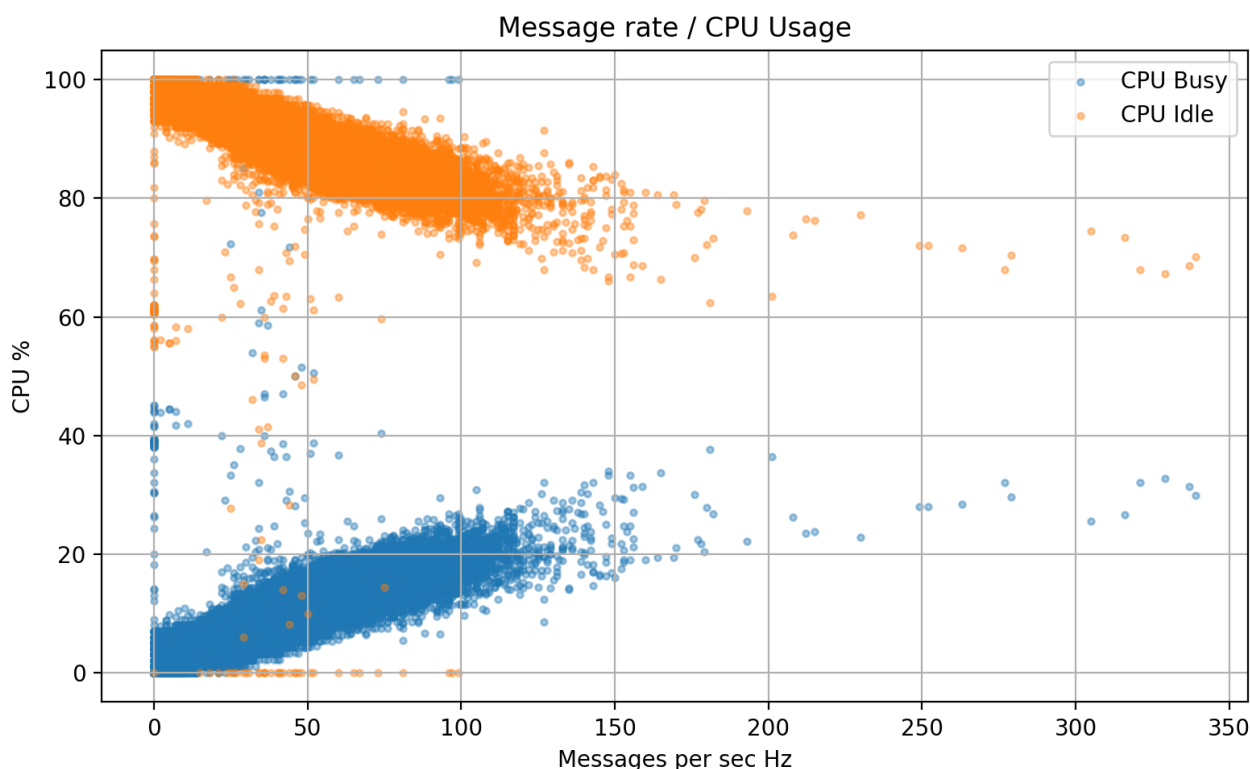
Ο ρυθμός εισόδου είναι εμφανώς bursty: ο μέσος όρος ήταν 39,29 Hz, ενώ η μέγιστη τιμή έφτασε τα 339 Hz. Παρά τις αιχμές, η μέγιστη καταγεγραμμένη πληρότητα του buffer ήταν μόνο 7% και στο 97,69% των δειγμάτων ο buffer ήταν άδειος τη στιγμή της μέτρησης. Αυτό δείχνει ότι ο Consumer προλάβει να επεξεργάζεται τα μηνύματα σημαντικά ταχύτερα από όσο συσσωρεύονταν στην ουρά στο συγκεκριμένο 24ωρο.



Σχήμα 3. Ρυθμός εισερχόμενων μηνυμάτων και πληρότητα του circular buffer.

3.2 Συσχέτιση ρυθμού μηνυμάτων και CPU

Η μέση χρήση της CPU ήταν 9,1% (median 8,82%). Υπάρχει σαφής θετική συσχέτιση μεταξύ traffic και CPU Busy. Όταν αυξάνονται τα μηνύματα ανά δευτερόλεπτο αυξάνεται η επεξεργασία JSON και μειώνεται αντίστοιχα το CPU Idle. Η μέγιστη τιμή 100% εμφανίστηκε μόνο σε 35 από τα 86.400 δείγματα (~0,04%) και όχι ως παρατεταμένη κατάσταση· το 95ο εκατοστημόριο της χρήσης CPU ήταν 16,16%. Επομένως το Raspberry Pi είχε σημαντικό περιθώριο επεξεργαστικής ισχύος στο μεγαλύτερο μέρος της εκτέλεσης.



Σχήμα 4. Συσχέτιση message rate με CPU Busy και CPU Idle.

4. Μακροχρόνια λειτουργία και ανθεκτικότητα δικτύου

Η συνεχής συλλογή 86.400 διαδοχικών δειγμάτων χωρίς timestamp gaps δείχνει ότι το πρόγραμμα παρέμεινε ενεργό για ολόκληρο το 24ωρο χωρίς segmentation fault ή διακοπή του περιοδικού Monitor. Ο circular buffer δεσμεύεται στατικά με σταθερό μέγεθος defined 5MB, άρα η βασική μνήμη της ουράς παραμένει σταθερή και δεν αυξάνεται με τον αριθμό των μηνυμάτων. Στον Consumer κάθε cJSON αντικείμενο που δημιουργείται με cJSON_Parse() αποδεσμεύεται με cJSON_Delete(), ενώ μετά από αποσύνδεση το WebSocket context καταστρέφεται με lws_context_destroy() πριν δημιουργηθεί νέο. Έτσι δεν υπάρχει δομή δεδομένων που να συσσωρεύεται απεριόριστα όσο περνά ο χρόνος και στο 24ωρο run δεν εμφανίστηκε συμπεριφορά αστάθειας.

Για στιγμιαία απώλεια δικτύου, το callback θέτει connection_closed=1 όταν προκύψει client_connection_error ή client_closed, όπου δεν παρατηρήθηκε κάποια τέτοια απώλεια μέσα στο 24ωρο. Ο Producer βγαίνει από το lws_service loop, καταστρέφει το προηγούμενο context, περιμένει 5 s και προσπαθεί ξανά να δημιουργήσει σύνδεση. Τα υπόλοιπα threads δεν τερματίζονται, επομένως το metrics_log.txt συνεχίζει να ενημερώνεται και μια προσωρινή αποσύνδεση εμφανίζεται ως χαμηλός ή μηδενικός ρυθμός μηνυμάτων μέχρι την επανασύνδεση. Η παρούσα υλοποίηση εξασφαλίζει αυτόματη επαναφορά της εφαρμογής, αλλά δεν αποθηκεύει ή ανακτά εκ των υστέρων μηνύματα που δεν παραδόθηκαν από το δίκτυο κατά το διάστημα της αποσύνδεσης.

5. Συμπεράσματα

Η 24ωρη εκτέλεση έδειξε ότι η υλοποίηση μπορεί να λειτουργήσει σταθερά για μεγάλο χρονικό διάστημα στο Raspberry Pi Zero W, επεξεργαζόμενη συνεχώς τη ροή δεδομένων του Bluesky Jetstream. Παρόλο που ο ρυθμός των εισερχόμενων μηνυμάτων παρουσίαζε έντονες μεταβολές και στιγμιαίες αιχμές έως 339 μηνύματα/s, η μέγιστη πληρότητα του circular buffer παρέμεινε μόλις στο 7% και η μέση χρήση της CPU περίπου στο 9,1%. Αυτό δείχνει ότι το σύστημα είχε σημαντικό περιθώριο επεξεργασίας για το φορτίο που παρατηρήθηκε.

Παράλληλα, η χρήση mutex και condition variable επέτρεψε τον ασφαλή συγχρονισμό μεταξύ των threads χωρίς race conditions ή busy waiting. Η περιοδική εκτέλεση του Monitor με `clock_nanosleep()` και absolute timing διατήρησε τη μέτρηση πολύ κοντά στο ιδανικό διάστημα του ενός δευτερολέπτου, με πρακτικά μηδενικό cumulative drift και χωρίς timestamp gaps κατά τη διάρκεια των 24 ωρών.

Συνολικά, η αρχιτεκτονική Producer-Consumer αποδείχθηκε κατάλληλη για τη συγκεκριμένη εφαρμογή πραγματικού χρόνου, καθώς διαχωρίζει τη λήψη, την επεξεργασία και την καταγραφή των δεδομένων σε ανεξάρτητα threads. Επιπλέον, η δυνατότητα αυτόματης επανασύνδεσης του Producer προσφέρει βασική ανθεκτικότητα σε προσωρινά προβλήματα δικτύου. Περιορισμός της παρούσας υλοποίησης είναι ότι μηνύματα που χάνονται κατά τη διάρκεια πραγματικής αποσύνδεσης δεν μπορούν να ανακτηθούν μετά την επανασύνδεση.